JellyGarden Match-3 — Technical & Visual Audit

Project: `J:/Unity Projects/Match3-Games`

Game: JellyGarden — "Match 3 Jelly Garden Kit" v2.5.2, Unity Asset Store productId **43260**, © Candy Smith

Engine: Unity **6000.5.10f1**

Audit type: Read-only discovery and documentation. No gameplay, art, scene, prefab or setting was changed to produce this document.

Audience: a senior UI/UX designer with no Unity production background.

READ THIS FIRST — THIS PROJECT IS NOT IN ITS SHIPPED STATE

Before this audit began, **an AI agent modified this project**. Those changes are uncommitted and are described throughout, always labelled. Do not read this report as a description of the product Candy Smith shipped.

What was added (all untracked in git — no history, so they are additions):

Path	What it is
`Assets/JellyGarden/Scripts/Juice/`	5 new C# files: VFX service, tuning config, easing, procedural textures, bloom bootstrap
`Assets/JellyGarden/Shaders/JuiceAdditive.shader`	The project's **only** custom shader
`Assets/JellyGarden/Resources/JuiceConfig.asset`	~45 Inspector tuning sliders
`Assets/JellyGarden/Resources/JuiceAdditive.mat`	Material for the new shader
`Assets/Settings/JG_URP_2D.asset`, `JG_2DRenderer.asset`	A Built-In to URP 2D render pipeline switch

What was edited in place (`git diff HEAD`): `Assets/JellyGarden/Scripts/Item.cs` **+395 / −45**, `Assets/JellyGarden/Scripts/LevelManager.cs` **+65 / −0**, plus `ProjectSettings/GraphicsSettings.asset` and `QualitySettings.asset`.

What was NOT touched: no scene, no prefab, no sprite, no font, no audio clip, no animation clip. CONFIRMED — `git status` shows `game.unity`, `main.unity` and every `.prefab` unmodified. **Every UI, art and animation finding in this report therefore describes the original shipped kit.**

Three defects in that added work are documented here (see 12.4 and 26): the bloom cannot render, the shader is not SRP-Batcher compatible, and the shader multiplies alpha twice.

Also not original, and not the AI agent's work: `Assets/GameRedesign/` (38 MB of orphaned concept art, 9.6) and `Assets/_Recovery/` (19.7).

---

1. Executive Summary

JellyGarden is a **2015-era commercial Unity Asset Store match-3 kit**, upgraded to Unity 6 but architecturally unchanged. It is complete and playable: 100 levels, four objective types, eight boosters, five special-piece combination rules, a world map, and ads/IAP/social hooks. As a foundation for a visual redesign it has real strengths and several structural constraints that will shape everything you can do.

The five things that matter most to a designer:

1. **The board is not UI.** Fruit and cells are world-space `SpriteRenderer` objects viewed by an orthographic camera; the HUD is a separate UGUI Canvas. The two scale by *different rules* — the board by screen **height**, the UI by screen **width**. On most phones they drift apart, and no amount of moving things in the scene fixes it. (Section 7.3)

2. **Feedback speed is capped by hard-coded waits, not by art.** `LevelManager.FallingDown()` contains at least four literal `WaitForSeconds` values per cascade wave plus 100 ms-granularity polling gates. **Floor is roughly 0.8 s per wave.** You can make every animation instant and the game will not get meaningfully faster until those numbers change. (Section 5.4)

3. **There is no 9-slicing and there are no sprite atlases.** All 225 sprites have `spriteBorder: {0,0,0,0}` and there is not a single `.spriteatlas` in the project. Panels and buttons cannot be resized without distorting, and every distinct sprite is its own draw call. (Sections 7.6, 19.2)

4. **Text is legacy `UnityEngine.UI.Text` — there is no TextMeshPro at all.** 57 Text components in the gameplay scene, roughly 167 across the project including prefabs. Outlines are faked by duplicating text geometry, in about 18 inconsistent variants. Nothing auto-sizes. (Section 7.4)

5. **The original core match effect was a 4-frame flip-book drawn *behind* the fruit.** `ItemExpl` sits at sorting order 0; the fruit sits at order 2. The puff was partly occluded by the very piece it was destroying. This is the mechanical reason the original destruction reads flat. (Section 13.1)

What is genuinely reusable: the level format (a readable text file per level, plus a full in-Editor level editor), the objective ScriptableObjects, the booster dispatch system, the special-piece combination matrix, and the map. (Section 22)

What is weakest: audio (no mixer, no pitch variation, no cascade escalation, one shared AudioSource — Section 14) and per-event feedback layering (Section 21).

---

2. How This Unity Project Is Built

Fact	Value	Evidence
Unity version	6000.5.10f1 (rev `3bd4f66ad299`)	CONFIRMED — `ProjectSettings/ProjectVersion.txt`
Kit version	2.5.2	CONFIRMED — `AssetOrigin` block in 677 `.meta` files
Product / company	`Match3 Games` / `DefaultCompany`	CONFIRMED — `ProjectSettings.asset:15-16`
Orientation	**Portrait only** (autorotate portrait + upside-down; both landscape flags 0)	CONFIRMED — `ProjectSettings.asset:11, 61-64`
Target device	iPhone **and iPad** (`targetDevice: 2`)	CONFIRMED — `ProjectSettings.asset:12`
Scripting backend	Android IL2CPP, iOS IL2CPP, Standalone Mono	CONFIRMED — `ProjectSettings.asset:859-865`
Bundle identifier	**empty** — not set for any platform	CONFIRMED — `ProjectSettings.asset:172`
Assembly definitions	**None** (zero `.asmdef` in the project)	CONFIRMED
Addressables / AssetBundles	**None**	CONFIRMED
Localization	**None**	CONFIRMED

2.1 Render pipeline — changed, and partly broken

- **Shipped state:** `m_CustomRenderPipeline: {fileID: 0}` — the **Built-In Render Pipeline**. CONFIRMED via `git show HEAD:ProjectSettings/GraphicsSettings.asset`.
- **Current state:** points at `Assets/Settings/JG_URP_2D.asset` (a URP 2D pipeline), on the default and all six quality levels. CONFIRMED.
- **Important nuance:** URP was **already installed** in the shipped project (`com.unity.render-pipelines.universal 17.5.0` in an unmodified `Packages/manifest.json`), and `Assets/Settings/` already contained committed `Mobile_RPAsset`, `PC_RPAsset`, `Mobile_Renderer`, `PC_Renderer` and

`UniversalRenderPipelineGlobalSettings` — Unity 6 URP-template scaffolding that was never wired up. The AI agent did not *introduce* URP; it **activated** it and added two new 2D-specific assets alongside the four now-unused template ones. CONFIRMED via `git ls-tree HEAD Assets/Settings/`.

> **Designer translation.** A "render pipeline" is the recipe Unity uses to turn a scene into pixels. This game was built for the old recipe. It now runs on the modern 2D recipe, which is what makes screen-wide glow possible in principle. Four leftover pipeline assets from the project template still sit in that folder unused, which is a housekeeping trap: it is not obvious which of the six is live.

Note also that part of the `GraphicsSettings.asset` diff (`serializedVersion 12 to 16`, removal of `m_LegacyDeferred`, addition of `m_BrgStripping` and similar) is a **Unity 6.5 editor upgrade artifact**, not an intentional edit. Only the `m_CustomRenderPipeline` line and the six quality-level entries are the deliberate pipeline switch. CONFIRMED.

2.2 Scenes — there are only two, and one holds nearly everything

#	Scene	Size	GameObjects	Role
0	`Assets/JellyGarden/Scenes/main.unity`	39.5 KB	**10**	Splash / logo / Play button
1	`Assets/JellyGarden/Scenes/game.unity`	787 KB	**261**	**Map + board + every popup**

CONFIRMED. `GUIEvents.cs:39` calls `SceneManager.LoadScene("game")` — the only transition out of the bootstrap scene.

There is no separate gameplay scene. `LevelManager.EnableMap(bool)` toggles two root objects (`LevelsMap`, `Level`) and destroys the board's children on the way back to the map. CONFIRMED.

> **Designer consequence.** Every popup, the HUD, the map and the board coexist in one 787 KB file. Two designers cannot edit different screens simultaneously without a merge conflict, and finding a specific popup means hunting through 261 objects.

Two dead scene references exist: `TestLevelGUI.cs:47` loads a scene named `demo` which is **not in the build settings**, and `Assets/Scenes/SampleScene.unity` is Unity template leftover. CONFIRMED.

2.3 The core class lives on the camera

`LevelManager` (about 98 KB of source) and `InitScript` are MonoBehaviour components on the GameObject named **`Main Camera`**. CONFIRMED by resolving `m_GameObject: {fileID: 511786103}` back to its `m_Name`. There is no "GameManager" object. `SoundBase` and `MusicBase` also re-parent themselves onto that camera at `Awake`.

2.4 No assembly definitions

Every C# file — roughly 120 gameplay scripts, all the dormant ads/IAP/PlayFab/GameSparks code, the 85 KB level editor, and the new juice layer — compiles into one `Assembly-CSharp`.

> **Designer consequence.** Every script edit recompiles the whole project, so iterating on a timing value *in code* is slow. **The fast path is `JuiceConfig.asset`** — its ~45 sliders are data, editable in the Inspector while the game runs, with no recompile.

---

3. Scene & System Architecture

```
main.unity  (10 objects)
   Logo / Loading / Play  --GUIEvents-->  SceneManager.LoadScene("game")
                                                    |
game.unity  (261 objects) <-------------------------+
   |
   +- Main Camera --- LevelManager (god class)  +  InitScript (economy)
   |                  SoundBase / MusicBase weld themselves here at Awake
   +- AI ------------ hint scanner + dead-board detector (the same code)
   +- LevelsMap ----- world map, spline path, 101 level nodes   [toggled]
   +- Level --------- the gameplay screen                      [toggled]
   |    +- CanvasBack  (Screen-Space Camera, order 0) - background image
   |    +- GameField   (WORLD SPACE)  - squares, blockers, fruit
   |    +- Canvas      (Screen-Space Camera, order 1) - HUD + booster bar
   |    +- CanvasScore (WORLD SPACE)  - floating score numbers
   |    +- TutorialManager - dim layer, hand, caption
   +- CanvasGlobal  (Screen-Space Camera, order 2) - ALL popups
```

The state machine is a property setter. Control flow is not driven by events. `LevelManager.gameStatus` (`LevelManager.cs:276-372`) is a large `if/else if` chain that, *on assignment*, drives menus, music, `Time.timeScale`, level init, ad checks and PlayerPrefs writes. **Assigning a state is the transition.** CONFIRMED.

Events exist but are near-vestigial. Eight `static` C# events are declared (`LevelManager.cs:264-274`); `EventsListener.cs` is the only subscriber, and they are raised **without null-guards**, so a missing listener throws a `NullReferenceException`. CONFIRMED.

Everything is a singleton: `LevelManager.THIS` *and* `LevelManager.Instance` (two statics for the same object), `AI.THIS`, `InitScript.Instance`, `MenuManager.Instance`, `SoundBase.Instance`, `MusicBase.Instance`, `InputManager.Instance`. Nothing can be previewed or tested in isolation.

Save system: `PlayerPrefs` only — an unencrypted local key/value file. Keys include `OpenLevel`, `Score<n>`, `Level.{000}.StarsCount`, `Gems`, `Lifes`, `RestLifeTimer`, `Music`, `Sound`, `Lauched` *(misspelled in the original)*. No cloud save is active. CONFIRMED.

---

4. How the Match-3 Board Works

4.1 A level is a text file

`Assets/JellyGarden/Resources/Levels/<n>.txt` — **100 files**, parsed by `LevelManager.ProcessGameDataFromString`. Complete example (`1.txt`):

```
MODE 3
SIZE 9/5
LIMIT 0/6
COLOR LIMIT 4
STARS 100/200/300
COLLECT COUNT 0/0
COLLECT ITEMS 0/0
10 10 10 10 10 10 10 10 10
10 10 20 20 10 10 10 10 10
10 10 20 20 10 10 10 10 10
10 10 10 10 10 10 10 10 10
10 10 10 10 10 10 10 10 10
```

Header	Meaning
`MODE n`	0 SCORE, 1 COLLECT, 2 INGREDIENT, 3 BLOCKS
`SIZE cols/rows`	Board dimensions, **columns first**
`LIMIT t/v`	t: 0 MOVES, 1 TIME. v: moves or seconds
`COLOR LIMIT n`	Fruit colours in play (3 to 6)
`STARS a/b/c`	Score thresholds for 1/2/3 stars
`COLLECT COUNT` / `COLLECT ITEMS`	Collectable quantities and which ones

Each grid cell is a **two-digit code**: first digit is the cell type, second the obstacle. `SquareTypes` = `NONE 0, EMPTY 1, BLOCK 2, WIREBLOCK 3, SOLIDBLOCK 4, DOUBLEBLOCK 5, UNDESTROYABLE 6, THRIVING 7`.

Verified cell-code usage across all 100 levels (9,252 cells total):

Code	Meaning	Count
`10`	plain playable cell	5,608
`14`	solid block	916
`00`	hole — nothing drawn, pieces fall past	889

20	jelly (1 hit)	698
50	double jelly (2 hits)	428
13	caged cell — the fruit inside cannot be dragged out	354
17	thriving / spreading blocker	199
23	jelly + cage	160

UNDESTROYABLE is supported by the code but **used by none of the 100 shipped levels**. CONFIRMED.

Level design distribution: BLOCKS 40 / INGREDIENT 24 / COLLECT 21 / SCORE 15; MOVES 89 / TIME 11; five colours in 54 levels, four in 22, six in 22, three in 2; **9 x 11 board in 67 of 100 levels**.

> **Designer translation.** Levels are plain readable text — you can open one in Notepad and see the board. There is also an 85 KB in-Editor level authoring tool (LevelMakerEditor.cs). This is the single most designer-friendly part of the project.

4.2 A fruit is a two-level object — and this matters

Assets/JellyGarden/Resources/Prefabs/Item.prefab:

- **Root** Item (tag Item): Transform, Item.cs, BoxCollider2D (1 x 1), Animator
- **One child** Sprite: Transform + SpriteRenderer, sorting order **2**, material Sprites-Default

Every hand-authored animation clip targets the CHILD (path: Sprite), never the root. CONFIRMED in destroy.anim and stop.anim curve bindings.

> **Designer translation.** The outer shell carries the touch collider and the animation controller; the inner child carries the picture. The authored animations move the *inner* picture, which leaves the *outer* shell free for code to animate independently. That split is genuinely useful — two systems can animate one fruit without overwriting each other.

The fruit sprite is Assets/JellyGarden/Textures/Items/item_01.png, 128 x 128 px at 100 pixels-per-unit. Board cell pitch measured at runtime is **0.82 world units**; the fruit is instantiated at localScale 0.6, so it renders 0.6 x 1.28 = **0.768 world units**, about 94% of the cell.

4.3 The cell model

Assets/JellyGarden/Scripts/Square.cs. One Square per grid cell, held in a flat array indexed row * maxCols + col. The predicates that matter to design:

- CanGoInto() — can a fruit occupy this cell (false for SOLIDBLOCK, UNDESTROYABLE, NONE, THRIVING)

- `CanGoOut()` — can the fruit here leave (false for WIREBLOCK). **This is what makes a caged fruit undraggable.**
- `IsNone()` — a hole; nothing is drawn and pieces fall straight past
- `DestroyBlock()` — pops one jelly layer, awards score, and **cascades to adjacent SOLIDBLOCK / THRIVING neighbours**, which is how solid blocks clear without a fruit ever sitting on them

No diagonal falling. `FallOut()` only ever consults `GetNeighborBottom()` and then straight down the same column. Pieces do not slide around overhangs. This is a hard constraint on level shapes. CONFIRMED.

---

5. Gameplay Execution Flow

5.1 Input to swap

1. **`Item.Update()`** (`Item.cs:1403`) — on touch-press, each fruit runs its own `Physics2D.OverlapPoint`; if it is the one hit, `dragThis = true`.

2. **`Item.CheckItemDrag()`** — computes `deltaPos = startPos - currentPos`.

3. **`Item.SwitchDirection()`** (`Item.cs:386`) — the swipe threshold is `Magnitude(deltaPos) > 0.1f` **in world units**, a tenth of a cell. Picks the dominant axis, resolves the neighbour square, grabs its item, and starts `Switching()`.

> **Designer warning.** The swipe threshold is a *world-space* distance, not a pixel or screen-percentage distance. Because the camera's orthographic size changes with device aspect (6.5 / 7.6 / 8.1 — see 7.3), **the effective swipe sensitivity differs between phones.** The number lives at `Item.cs:389` and is not resolution-independent.

Note that `LevelManager.Update()` *also* polls input and runs its own `OverlapPoint`, and `Item.OnMouseDown/Up/Drag` also still exist as dead code (they only fire under the old input handler, which is disabled). There are effectively three input paths in the file, two of them live and one dead. CONFIRMED.

5.2 Swap validation — the model is swapped first, then tested

`Item.Switching()` (`Item.cs:433-628`):

```
DragBlocked = true                                    // locks the whole board
neighborSquare.item = this;  square.item = switchItem;   // SPECULATIVELY swap
matchesHere           = neighborSquare.FindMatchesAround().Count
matchesInNeithborItem = square.FindMatchesAround().Count
```

The board model is swapped **before** any test, both cells are asked "would you have a match here", and it is undone if not. Bonuses are then added: +1 if either piece is a colour bomb, +1 if both are special, forced to 0 if both are ingredients.

The Hand booster is the escape hatch — when active it bypasses the no-match rejection *and* skips the move decrement. CONFIRMED (`Item.cs:466, 521-528`).

5.3 Match detection — there are two independent finders

This is a significant architectural quirk.

Finder	Where	Used for
`Square.FindMatchesAround()`	`Square.cs:159`	Speculative "would this be a match" — swap validation, and the cascade loop's **exit** test
`CombineManager.GetCombine()`	`Scripts/System/CombineManager.cs`	**Authoritative** — decides what actually dies and which special is created

They use different algorithms. INFERRED: this is a plausible source of the occasional extra empty cascade wave that costs a fraction of a second of nothing happening.

Special-piece creation is decided in `CombineManager.SetNextItemType(Combine)`:

Condition	Result
5 or more in a line	**BOMB** (colour bomb)
L / T / cross (both axes >= 3)	**PACKAGE** (area bomb)
exactly 4	**STRIPED**, horizontal or vertical depending on the player's drag direction
plain 3	none

A **second, parallel creation path** exists inside `Item.Switching()` using different match counts. So two places decide "you earned a striped piece". INFERRED: this redundancy is why specials occasionally look inconsistent with the visible match shape.

5.4 The cascade loop — and the timing ceiling

`LevelManager.FallingDown()` (`LevelManager.cs:1567-1888`) is the entire resolve-the-board pipeline in one coroutine. Per cascade wave:

Step	What	Cost
1	`WaitForSeconds(0.1f)` before any match search	**100 ms hard**

2	`CombineManager.GetCombine()`	—
3	`combo++`	—
4	For each item: if it is a **special**, `WaitForSeconds(0.1f)` first; then `DestroyItem()`	**+100 ms per special**
5	If the player just earned a special, `WaitForSeconds(0.5f)`	**+500 ms, once per move**
6	**`while (!IsAllDestoyFinished()) WaitForSeconds(0.1f)`**	**THE GATE**
7	20 full board passes of `FallOut()` in one frame (model only)	—
8	`WaitForSeconds(0.2f)` (skipped if pieces were already sliding)	**200 ms**
9	All pieces animate their fall simultaneously	—
10	`WaitForSeconds(0.2f)`	**200 ms**
11	`GenerateNewItems()` refill, `RegenMatches()`	—
12	`WaitForSeconds(0.1f)` then poll `IsAllItemsFallDown()` every 100 ms	**100 ms+**

The destroy gate is the single most important timing mechanism in the game

```
bool IsAllDestoyFinished() {
    GameObject[] items = GameObject.FindGameObjectsWithTag("Item");
    foreach (GameObject item in items) {
        Item c = item.GetComponent<Item>();
        if (c == null) return false;
        if (c.destroying && !c.animationFinished) return false;
    }
    return true;
}
```

> **Designer translation — this is the rule that governs how fast your game can ever feel.**

>

> 1. **The slowest death animation on screen sets the pace for the whole board.** If one fruit's pop takes 250 ms, nothing falls for 250 ms — even in a corner of the board with nothing to do with that match.

> 2. **The gate is polled at 100 ms granularity, so the true wait always rounds *up* to the next 100 ms tick.** A 160 ms animation costs 200 ms of gate time. Shaving an animation from 260 ms to 210 ms buys you nothing; shaving it to 190 ms buys a full 100 ms.

> 3. `GameObject.FindGameObjectsWithTag` allocates a fresh array of about 99 GameObjects and calls `GetComponent` on each, **ten times a second, in four separate places** in the pipeline — during exactly the moments you most want a smooth frame rate.

Floor on one cascade wave, from the hard waits alone: roughly 0.6 s, plus polling, so about 0.8 s. A four-deep cascade cannot resolve in under roughly 3.5 seconds no matter how fast the art is.

If the brief is "make the game feel snappier", these `WaitForSeconds` literals are the ceiling, not the animation curves.

5.5 One complete traced move

Player swipes a fruit into a valid horizontal 3-match, on a MOVES level:

#	Class.Method	Location	What happens
1	`InputManager.OnTouchPerformed`	`InputManager.cs:230`	Input System fires; `wasTouchPressed = true`
2	`Item.Update`	`Item.cs:1407`	`OverlapPoint` hits this fruit; `dragThis = true`
3	`Item.SwitchDirection`	`Item.cs:386`	Threshold passes; resolves neighbour; starts `Switching()`
4	`Item.Switching`	`Item.cs:433`	`DragBlocked = true`; speculative model swap
5	`Square.FindMatchesAround` x2	`Square.cs:159`	3 and 0
6	`Item.Switching`	`Item.cs:466`	`rejectSwap` = false
7	`Item.Switching`	`Item.cs:492`	Both fruits travel, eased, **130 ms** *(AI-edited; originally linear, about 200 ms)*
8	`Item.Switching`	`Item.cs:519`	`Limit--`, `moveID++`, ownership reassigned
9	`LevelManager.FindMatches`	`:1479`	`StartCoroutine(FallingDown())`

10	`FallingDown`	`:1587`	**`WaitForSeconds(0.1f)`**
11	`CombineManager.GetCombine`	`CombineManager.cs:29`	One combine, hCount 3
12	`FallingDown`	`:1675`	Plain fruit, so **no** per-item wait; `DestroyItem(true)` on all three
13	`Item.DestroyCor`	`Item.cs:1054`	Plays destroy SFX; starts the pop
14	`LevelManager.PopupScore`	`:2211`	"+10" floats up in the fruit's colour
15	`LevelManager.CheckCollectedTarget`	`:575`	Objective bookkeeping; may fly a copy to the HUD
16	`Item.DestroyCor`	`Item.cs:1110`	Waits on `animationFinished`
17	`Square.DestroyBlock`	`Square.cs:270`	Clears jelly underneath if present
18	`FallingDown`	`:1728`	**The destroy gate** — releases on the next 100 ms tick
19	`Square.FallOut` x20 passes	`Square.cs:212`	Model-only settle
20	`FallingDown`	`:1747`	**`WaitForSeconds(0.2f)`**
21	`Item.FallingCor`	`Item.cs:735`	Every displaced fruit animates down
22	`FallingDown`	`:1770`	**`WaitForSeconds(0.2f)`**
23	`GenerateNewItems`	`:1243`	New fruit spawns 3 units above its cell
24	`FallingDown`	`:1773`	`WaitForSeconds(0.1f)` + fall polling
25	`FallingDown`	`:1884`	**`DragBlocked = false`** — playable again
26	`AI.CheckPossibleCombines`	`AI.cs:106`	Waits 1 s, scans, then waits another 1 s before showing a hint

Total floor from finger-lift to playable: about 0.93 s, of which roughly 0.8 s is hard-coded `WaitForSeconds`, not animation.

5.6 Win, lose, hint and shuffle

- **Win/lose** (`CheckWinLose`, `:661`): you win the instant all objectives are done. If you run out of moves with all objectives done you *additionally* need `Score >= star1` — and if you do not have it, **the code sets no state at all**, a silent stall. CONFIRMED from the structure; INFERRED that it is reachable.
- **The win animation can run 12+ seconds.** `PreWinAnimationsCor` converts up to 15 leftover moves into stripes at 0.5 s each, then clears them at about 1.1 s each, then shows a 3 s banner.
- **The hint system and the dead-board detector are the same code** (`AI.cs`). It fires on a fixed clock — 1 s in `CheckPossibleCombines` plus 1 s in `showTipCor`, so **the hint appears about 2 s after the board settles, whether or not the player is thinking.** There is no idle-timeout concept.
- **The dead-board test only understands plain colour matches** (13 hard-coded shape templates on `item.color`). A board whose only legal move requires a colour bomb or a stripe would be declared dead. INFERRED from the template code.
- **"Shuffle" is not a shuffle.** `ReGenLevel` destroys every fruit and generates a fresh board. Blockers, jelly and progress survive; the fruit does not.

5.7 Boosters

`BoostType` = `ExtraMoves, Packages, Stripes, ExtraTime, Bomb, Colorful_bomb, Hand, Random_color, None`. Activation is dispatched by the **setter** of `LevelManager.ActivatedBoost`.

Booster	When	Mechanism
ExtraMoves / ExtraTime	Instant	`Limit += 5` (MOVES) or `+= 30` (TIME)
Packages / Colorful_bomb / Stripes	Pre-level	`GetRandomItems()` then set `NextType`
Bomb	In-level, tap-targeted	Spawns `Effects/bomb`; its animation event destroys the item
Random_color	In-level, tap-targeted	Spawns `Effects/random_color_item`; recolours the 3x3 neighbourhood
Hand	In-level, modal	Allows an illegal swap and skips the move cost

---

6. What the Game Pieces Are Made From

Everything on the board is a flat 2D sprite. There is no 3D geometry anywhere in this project — no `.fbx`, `.obj` or `.blend` files exist. CONFIRMED.

Piece	Prefab	Component	Source art	Sorting

Fruit	`Resources/Prefabs/Item.prefab`	`SpriteRenderer` on child `Sprite`	`Textures/Items/item_01..06.png`, 128x128 @ 100 PPU	Default / **2**
Board cell	`Resources/Prefabs/Square.prefab`	`SpriteRenderer`	checkerboard sprites	Default / 0
Jelly	`Resources/Prefabs/Block.prefab`	`SpriteRenderer`		Default / 1
Cage	`Resources/Prefabs/WireBlock.prefab`	`SpriteRenderer`		Default / 3
Solid block	`Resources/Prefabs/SolidBlock.prefab`	`SpriteRenderer`		Default / 3
Spreading blocker	`Resources/Prefabs/ThrivingBlock.prefab`	`SpriteRenderer`		Default / 3
Undestroyable	`Resources/Prefabs/UndestrBlock.prefab`	`SpriteRenderer`		Default / 3

Special pieces are not separate prefabs. A striped, wrapped or colour-bomb piece is the **same `Item` prefab with a different sprite swapped in** — `Item.cs` holds `Sprite[] items`, `List<StripedItem> stripedItems`, `Sprite[] packageItems`, `Sprite[] bombItems`, and `ChangeType()` assigns from those arrays. CONFIRMED.

> **Designer translation.** To restyle a fruit you replace one 128x128 PNG. To restyle a *striped* fruit you replace an entry in the `stripedItems` list on the Item prefab. You never touch a separate prefab. The colour bomb uses `color = 555`, a sentinel that stops it ever colour-matching.

---

7. What the UI Is Made From

7.1 Framework: UGUI, no UI Toolkit, no TextMeshPro

CONFIRMED: zero `.uxml` / `.uss` / `.tss` files; `com.unity.ugui 2.5.0` present; **no TextMeshPro package and no TMP font asset anywhere**.

Component census for `game.unity` (GUID-resolved):

Component	Count

`UnityEngine.UI.Image`	156
`UnityEngine.UI.Text` (**legacy**)	57
`UnityEngine.UI.Outline`	53
`UnityEngine.UI.Button`	45
`Counter_` (project)	37
`AnimationManager` (project)	19
`BoostIcon` (project)	9
`GraphicRaycaster` / `CanvasScaler` / `Canvas`	5 each
`GridLayoutGroup`	5
`SpriteRenderer` (world, not UI)	2

Notable absences — all zero in `game.unity`: `Mask`, `RectMask2D`, `CanvasGroup`, `ScrollRect`, `Slider`, `Toggle`, `LayoutElement`, `HorizontalLayoutGroup`, `VerticalLayoutGroup`, `ContentSizeFitter`, `AspectRatioFitter`.

> **That absence list is the single most important UI fact in this report.**

> - **No scrolling anywhere.** The gem shop, booster shop and leaderboard are fixed hand-placed rows. Adding a fifth pack means repositioning everything by hand.

> - **No clipping.** Nothing can be masked to a rounded frame or a scroll window.

> - **No fade groups.** A popup cannot be faded as a unit by changing one alpha; each element must be animated separately.

> - **Almost no auto-layout.** Five `GridLayoutGroup`s (the objective rows) are the only exception; every other position is a hand-typed pixel coordinate.

7.2 Canvases

Canvas	Path	Render mode	Layer/order	Reference res	Match

`CanvasGlobal`	`/CanvasGlobal`	Screen Space – Camera	UI / **2**	**800 x 1280**	0 (width)
`Canvas`	`/Level/Canvas`	Screen Space – Camera	UI / **1**	**720 x 1280**	0 (width)
`CanvasBack`	`/Level/CanvasBack`	Screen Space – Camera	Default / 0	720 x 1280	0 (width)
`CanvasScore`	`/Level/CanvasScore`	**World Space**	UI / 0	800 x 600 (ignored)	—
`CanvasTutorial`	`/Level/.../CanvasTutorial`	**World Space**	Default / 8	800 x 600 (ignored)	—

`main.unity` has one canvas in Screen Space – Overlay with reference resolution 720 x 1280 and **match 0.379** — the only place that value occurs in the project.

Two problems here, both CONFIRMED:

1. **The reference resolutions disagree.** Popups are designed against 800 x 1280, the HUD against 720 x 1280. Because both scale by width, **popups render about 11% smaller relative to the screen than the HUD does.** Artwork of the same source pixel size will not appear the same size. This is almost certainly unintentional drift.

2. **Two canvases use an 800 x 600 landscape reference resolution inside a portrait-only game.** They are World Space so the value is ignored, but it is a trap for anyone who changes their render mode.

7.3 The Canvas / World split — the most important composition fact

The board is not UI. It is world-space `SpriteRenderer` objects seen by an orthographic camera. The HUD is UGUI. **They scale by different rules:**

- The **board** is height-locked: `orthographicSize` fixes how many world units fit vertically.
- The **UI** is width-locked: `matchWidthOrHeight: 0` scales purely by screen width.

And the board's size is chosen by **exact float equality against a rounded aspect ratio** (`LevelManager.cs:411-443`, original code):

```
float aspect = (float)Math.Round((float)Screen.height / (float)Screen.width, 2);
GetComponent<Camera>().orthographicSize = 6.5f;              // default
if (aspect == 2.06f)      orthographicSize = 7.6f;           // 2960:1440
else if (aspect == 2.17f) orthographicSize = 8.1f;           // iPhone X
```

> **Designer warning — this is the highest-risk item in the whole layout system.** There are exactly **three hard-coded device buckets and no interpolation**. A phone at 2.22 (20:9, extremely common today), 2.00, 2.11,

2.16, or an iPad at 1.33 — and `targetDevice: 2` explicitly includes iPad — all fall through to the untuned 6.5 default. On those devices a 9 x 11 board may not fit the screen properly, and the gaps between HUD, board and booster bar will differ from every mockup. This cannot be fixed by moving things in the scene.

Safe area is handled on exactly one object. `SafeArea.cs` appears **once** in `game.unity` and **zero** times in `main.unity`. It runs only at `Awake` and does not respond to rotation — and rotation *is* enabled (portrait + portrait-upside-down), so flipping the phone 180 degrees leaves the notch inset pinned to the wrong edge. CONFIRMED.

Sorting layers. Only two exist, `Default` then `UI`:

Layer / order	Content
Default / 0	Background image; board squares and blocks
Default / 1	Jelly
Default / 2	**Game pieces**
Default / 3	Blockers; tutorial dim layer
Default / 7-8	Tutorial hand and caption
UI / 0	Floating score numbers (World Space canvas)
UI / 1	HUD, booster bar, praise word art
UI / 2	All popups

> **Practical consequence.** Because `UI` is a whole sorting layer above `Default`, **nothing on the board can ever draw over the HUD**, no matter what order numbers you use. A design where a collected fruit flies up *over* the score bar is not achievable by reordering — the flying object must be re-parented onto the canvas or moved to the UI layer. The game already does exactly that for collected objectives (`AnimateCollectedItem` copies the sprite onto sorting layer "UI", order 1).

7.4 Text — legacy, fake-bolded, and scaled by transform

57 `Text` components in `game.unity`, zero TextMeshPro. Adding prefabs, the real surface is about **167 Text components** project-wide.

Fonts:

Font	Size	Uses in `game.unity`	Where
`Candy Script_0.ttf`	313 KB	**51** (+105 in prefabs)	The house font — every score, counter, level number, price
`FONTH__.TTF`	35 KB	5	Life-shop labels only, **all on inactive objects**
`BorisBlackBloxx.ttf`	53 KB	1 (+4 in prefabs)	Tutorial caption, floating score numbers

Three consequences of legacy Text:

1. **Text sharpness is resolution-dependent.** Legacy Text rasterises glyphs at the exact pixel size requested; TextMeshPro stores outlines and stays crisp at any size.

2. **There is no built-in outline.** The candy-style stroke on every number is faked by duplicating the text mesh — the stock `Outline` component makes four shifted copies, so **every outlined label costs 4x its triangle count**, applied 53 times.

3. **Nothing auto-sizes.** All 57 Texts have `m_BestFit: 0`. Long localised strings will overflow rather than shrink.

The huge-font-then-shrink pattern. HUD numbers are authored at enormous sizes in enormous rectangles, then shrunk by the transform:

Element	Rect	Font size	localScale
Score	779 x 146	**120**	**0.3457, 0.3457, 0.6915** (non-uniform)
Moves / timer	435 x 228	50	0.885
Level number	435 x 228	120	0.3282
Booster counts (x5)	435 x 228	80	0.3283

> **Designer translation.** *The number you type into "Font Size" is not the size you see.* Effective size is roughly font size x scale — the Score's 120 pt at 0.3457 reads as about 41 pt. And the scale is **overwritten at runtime**: `Counter_.cs` forces `localScale = Vector3.one` for a moves counter and `Vector3.one * 0.68f` for a timer, so **the moves counter and the timer are different sizes at runtime, and neither matches what the scene shows in the editor.** `Counter_.cs` also recolours the timer red below 30 seconds.

There are seven authored font sizes (100 x22, 120 x14, 80 x9, 140 x4, 150 x3, 110 x3, 50 x2), each further modified by a per-object scale, so the effective type scale is effectively arbitrary. There are about **18 distinct outline configurations**, several clearly the result of dragging a slider.

A better outline component exists — `NicerOutline.cs` makes eight shifted copies for a rounder stroke — but it is used in **exactly one prefab** and never in either scene. CONFIRMED.

7.5 The HUD, element by element

Everything sits under `/Level/Canvas/Panel` (full-screen stretch, carries `SafeArea.cs`). Note there are **three nested GameObjects named `Panel`**; the one carrying the HUD plate Image is the innermost.

Element	Component	Source sprite	Native	Rect
HUD bar plate	`Image` (Simple)	`menu/game_menu/gui.png`	800 x 182	**800 x 182 — 1:1**
Score number	`Text` + `Outline`	—	—	779 x 146 @ 0.346
"SCORE" label	`Image`	`score_txt.png`	106 x 47	1:1 — **baked word art, cannot be localised**
Moves / timer	`Text` + `Outline` + `Counter_`	—	—	scale overwritten at runtime
"MOVES" label	`Image` + `MovesLabel`	`moves_txt.png`	109 x 47	swaps to `timer_txt.png` on timed levels
"TARGET" label	`Image`	`target_txt.png`	114 x 56	baked word art
Star progress track	`Image`	`gui_for_star_bar_01.png`	284 x 36	1:1
Star progress fill	`Image` **(Filled)** + `ProgressBarScript`	`star_bar.png`	278 x 30	**the only non-Simple project Image in the game**
Stars 1-3	`Image`	`star_for_bar_01/02.png`	41 x 40	**positions recomputed at runtime**
Pause button	`Image` + `Button`	`pause_button.png`	69 x 69	1:1
Objective row	`TargetsGroup` + `GridLayoutGroup`	runtime-populated	—	the only auto-layout in the HUD

Booster bar plate	`Image`	`gui_bottom.png`	800 x 171	1:1
5 booster buttons	`Image` + `Button` + `Animator` + `BoostIcon`	`Boosts/boost_*.png`	—	125 x 124 each

Every HUD sprite checked renders 1:1 with its source. There is no visible art distortion in the HUD today.

The star markers move at runtime. `ProgressBarScript.PrepareStars()` repositions stars 1 and 2 horizontally in proportion to that level's score thresholds. Their positions in the scene are placeholders — do not treat them as the design.

The background changes every 20 levels. `Background.cs` picks one of five sprites from `Textures/backgrounds/` based on `currentLevel / changeBackgoundEveryLevels`.

7.6 Nine-slicing: there is none

A grep for any non-zero `spriteBorder` across every `.meta` in `Assets/JellyGarden/Textures/` returns zero results. All 225 sprites have `spriteBorder: {0,0,0,0}`.

Of the 156 `Image` components in `game.unity`: **141 Simple, 14 Sliced, 1 Filled, 0 Tiled** — and **all 14 "Sliced" ones point at Unity's built-in default UI sprite**, not at project artwork. They are the invisible full-screen backing rectangles behind popups.

> **This is the hard constraint on the whole redesign.**

> - **You cannot resize any button, panel or bar without distorting its corners.** A wider score chip or a taller popup requires a **new PNG**, not a slider tweak.

> - Conversely, converting the key frames (`gui.png`, `gui_bottom.png`, `gui_for_star_bar_01.png`, `x_button.png`, the popup banners) to 9-sliced is a **cheap, high-leverage change**: set four border values in the importer and switch the Image type to Sliced, and those pieces become freely resizable.

>

> One counter-example exists: `Assets/GameRedesign/Resources/HUD_Card.png` has `spriteBorder: {165, 140, 165, 185}` — the only 9-slice border in the project, in the orphaned concept-art folder. Someone already understood the technique.

7.7 Buttons

45 `Button` components. Transition types: **26 Colour Tint, 19 Animation, 0 Sprite Swap.**

Of the 26 colour-tint buttons, **25 use Unity's stock default tint** (highlighted 0.96, pressed 0.784). The odd half-grey values with `m_ColorMultiplier: 2` sit almost entirely on the **19 Animation-transition buttons where the colour block never executes** — they are inert leftovers.

> **Designer takeaway.** Press feedback is currently either Unity's default grey dim or a hand-authored Animator clip; there is no consistent, designed button state system. And the button Animator itself is broken — see 10.4.

---

8. Prefab Architecture

Prefabs are Unity's reusable object templates. This project uses them lightly — most UI lives directly in the scene rather than in prefabs, which is why the gameplay scene is 787 KB.

Prefab	Path	Contains	References externally
`Item`	`Resources/Prefabs/Item.prefab`	Root: `Item.cs`, `BoxCollider2D`, `Animator`. Child `Sprite`: `SpriteRenderer` (order 2)	`Item.controller`; 6 fruit sprites + striped/package/bomb sprite arrays as on the component
`Square`	`Resources/Prefabs/Square.prefab`	`SpriteRenderer` (order 0)	checkerboard sprites
`Block` / `SolidBlock` / `WireBlock` / `UndestrBlock` / `ThrivingBlock`	`Resources/Prefabs/`	`SpriteRenderer`	blocker sprites
`ItemExpl`	`Resources/Prefabs/Effects/ItemExpl.prefab`	`SpriteRenderer` (**order 0**), `Animator`, `ItemAnimEvents`	`ItemDestroyController.contr` 6 flip-book clip sets
`Firework`	`Resources/Prefabs/Effects/Firework.prefab`	**1 ParticleSystem**, `ParticleSorting` (order 3)	`package_particle.mat`
`Replace`	`Resources/Prefabs/Effects/Replace.prefab`	**2 ParticleSystems**; `ParticleSorting` (order 3) on the **root only**	`ElectricBolt_Add 1.mat`, `Bubble.mat`
`CircleExpl`	`Resources/Prefabs/Effects/CircleExpl.prefab`	Animator-driven sprite	

`bomb` / `random_color_item`	`Resources/Prefabs/Effects/`	Booster overlay, `BoostAnimation`	
`PopupScores`	`Resources/Particles/PopupScores.prefab`	UI `Text` + `Outline`, legacy `Animation`	`flyupText.anim`, `popupText.an`
`StripesEffect`	`Resources/Particles/StripesEffect.prefab`	stripe beam visual	
`5sec`	`Resources/Prefabs/5sec.prefab`	"+5 seconds" flyer for TIME levels	`5secfly.anim`
`LevelsMap`	`Prefabs/Prefabs/LevelsMap.prefab`	**102 Canvases** (101 of them world-space per-level number canvases)	103 uses of the house font
`SoundBase`	`Prefabs/SoundBase.prefab`	one `AudioSource`, `SoundBase.cs` with ~25 clip fields	all SFX clips

A note on `Replace.prefab` (CONFIRMED defect): it has two particle systems, but only the **root** carries the `ParticleSorting` component that lifts it to order 3. The child named `Bubble` has its own renderer at **order 0**, and `ParticleSorting` only affects its own GameObject. **So half of the colour-bomb effect renders behind the fruit.** This is the same class of bug as the original destroy flip-book.

---

9. Visual Asset Library

9.1 Headline numbers

- **225 image files** in `Assets/JellyGarden/` (224 PNG + 1 TGA). 218 imported as Sprites.
- **224 of 225 at 100 pixels-per-unit**; one at 200.
- **All 225 have `spriteBorder` = zero** (no 9-slicing).
- **192 are non-power-of-two** dimensions.
- **Zero Sprite Atlases** anywhere in the project.
- **Zero 3D models** — no `.fbx`, `.obj`, `.blend`.
- **Zero PSD, SVG or AI files** — there are no layered or vector sources in the project.
- **3 fonts**, no TextMeshPro assets.
- **8 materials.**
- **35 audio files.**

9.2 Sprite packing — legacy tags, no atlases

`m_SpritePackerMode: 2` is set, and sprites carry legacy packing tags: 115 have no tag, 51 are tagged `Pack1`, 24 `Pack3`, 15 `Pack2`, 6 `Game`. But **no `.spriteatlas` asset exists**, so the modern atlas system is not in use.

> **Designer translation.** An atlas packs many small images into one big sheet so the GPU can draw them in a single batch. Without one, each distinct sprite tends to cost its own draw call. With 156 Image components and 225 sprites, this is the project's most obvious and cheapest performance win — and it is purely an import-settings change, requiring no art rework.

9.3 Compression and sizes

`textureCompression`: 194 sprites at 0 (uncompressed/normal quality), 27 at 1. `maxTextureSize`: 194 at the default 2048, 15 at 1024, one at 512, one at 256. There are **no per-platform (Android/iOS) texture overrides**.

9.4 Fonts

`Candy Script_0.ttf` (313 KB), `BorisBlackBloxx.ttf` (53 KB), `FONTH__.TTF` (35 KB). All stock importers, `fontSize: 16`, `fontRenderingMode: 0` (Smooth).

Housekeeping note: `Assets/JellyGarden/Fonts/Materials/` and `Fonts/Textures/` contain **no font assets** — they hold `snow.mat`, water materials, `Foam.png`, `Ripple.png`, `Splash_4Frames.png`. Misfiled particle art.

9.5 Orphaned assets in the live kit

At least ten assets return zero references across the project. Notable: `PackageExpl.prefab`, `Splash.prefab` and `Splash 1.prefab` (about 584 KB each, five particle systems each), `item_01.controller` and its `destroyItem.anim`, `bomb_expl.anim`, and five audio clips (about 747 KB) including a **`Combo 01.wav` that is exactly the asset a cascade-escalation system would need**.

9.6 `Assets/GameRedesign/` — 38 MB of orphaned concept art

This folder is **not part of the shipped kit and not the AI agent's work**. Dated 6 September.

- **52 files: 35 PNG + 17 JPG.** No scripts, no prefabs, no scenes.
- **Every one of its 65 GUIDs returns zero references** across all `.unity`, `.prefab`, `.asset` and `.mat` files, and the string "GameRedesign" appears in no `.cs` file. **Nothing in the game uses any of it.** CONFIRMED.
- Contents: a **king character** with six named expressions (`King_Neutral`, `King_Celebrate`, `King_Concerned`, `King_Impressed`, and others), **HUD concepts** (`HUD_Card`, `HUD_Medallion`, `HUD_Portrait`, `HUD_Rail`, each with `_v1`/`_v2` iterations), a **`Coin_Gold`** reward asset, and two **reference screenshots** (`RefSource/RefGame.png`, `RefKing.png`).
- **Five folders are completely empty:** `Prefabs/`, `Documentation/`, `Runtime/`, `VFX/`, `Character/Turnaround/`.
- The 21 PNGs in `GameRedesign/Resources/` are **byte-identical duplicates** of files in `UI/` and `Reward/` — but their `.meta` import settings differ, so they are **not interchangeable**.

Two things a designer must know about this folder:

1. **Because 21 of those PNGs sit in a folder literally named `Resources`, Unity will force-include them in the built app even though nothing references them.** Anything in a `Resources` folder ships whether used or not.

2. Two of the reference images (`RefGame.png`, `RefKing.png`) are imported as **`textureType: 0` (Default, not Sprite)** — they cannot be dropped onto a UGUI Image without changing their import settings first.

> This looks like an earlier, abandoned redesign exploration: a mascot character with an emotional range, and several HUD treatments. If a redesign is planned, **this folder is the most relevant existing design thinking in the project** — and the six king expressions imply an intended reaction system (celebrate / concerned / impressed) that the game has no code for.

9.7 What is NOT in this project

For a designer scoping a redesign, the absences are as informative as the contents:

Absent	Consequence
Sprite Atlases	Every sprite risks its own draw call
9-slice borders	Nothing can be resized without distortion
TextMeshPro	Text quality is resolution-dependent; no gradients, no proper outlines
3D models / meshes	Everything is flat sprites
PSD / SVG / AI sources	**There are no editable source files** — only flattened PNGs
Localization	Text is baked into the scene, and some labels are baked into PNGs
Scroll views, masks, canvas groups	No scrolling, clipping or unit-fading is possible without adding components

The absence of layered source files is worth emphasising: **every piece of UI art in this project is a flattened bitmap.** Re-colouring a button or changing a bevel means repainting the PNG.

---

10. Animation Architecture

10.1 Five different animation systems coexist

Do not assume the Animator drives everything. The real picture:

System	Where used	Notes
Animator + Animator Controller	Fruit (`Item.controller`), destroy flip-book, buttons, boosters	42 `.anim` clips, 7 controllers; 21 `Animator` components in `game.unity`
Legacy `Animation` component	Popups, banners, score popups	**36 `!u!111` Animation components** in `game.unity`; **19 of the 42 clips are `m_Legacy: 1`**
Coroutine + `Vector3.Lerp`	Swap, fall, idle breathing, tutorial hand, collected-item fly	The bulk of *gameplay* motion
Runtime-generated `AnimationClip`	Board entrance (`AnimateField`), collected-item fly curve	Built in C# with hard-coded keyframes — **not editable in the Animation window**
Code-driven juice *(AI-added)*	Squash, pop, landing, hit-stop	Driven from `JuiceConfig.asset`

> **Designer translation.** Two of these — Animator and legacy `Animation` — are parallel systems doing the same job, a legacy of the kit's age. Popups use the old one; buttons use the new one. If you ask "where do I change this animation?", the honest answer is "it depends which of five systems owns it", and this report's inventory (`08_Animation_Inventory.csv`) is the map.

No tween library is installed. No DOTween, LeanTween, PrimeTween or iTween anywhere. CONFIRMED. Timeline is installed but **no `.playable` asset exists**, so it is unused.

10.2 Animation events couple art timing to game logic

Twelve clips carry animation events that call back into C#. The critical one:

- `destroy.anim` fires **`SetAnimationDestroyingFinished` at t = 0.250 s** — and that is what releases the cascade gate described in 5.4.

> **Designer warning.** This means **the length of an art file is a gameplay timing value.** Retiming `destroy.anim` changes how fast the whole board resolves. An animator who shortens that clip to make the pop snappier is, without knowing it, changing the game's pacing. Other event-bearing clips: `appear`, `disappear`, `horr_appearing` (fire `SetAppeared`), `banner`, `bannerComplete`, `bannerFailed`, `Menu`, `floating_word`, `bomb_boost`, `random_color_boost`, `destroy_package`.

10.3 Clips animate the child, code animates the root

Verified: every `Item` clip's curves carry `path: Sprite` — they animate the **child**. The idle breathing coroutine and the new juice code animate the **root**. This is why both can run at once without fighting.

10.4 Two broken animation states, both CONFIRMED

1. **The button state machine cannot reach two of its five states.** `button_controller.controller` has `Normal`, `Highlighted`, `Pressed`, `Disabled` and `button_idle`. Every state's transition list is empty, and `AnyState` has only three transitions: `Normal->Normal`, `Pressed->Pressed`, `Disabled->Disabled`. **Nothing can ever reach `Highlighted` or `button_idle`.** The 1.03 s looping `button_idle` clip never plays.

2. **The score popup animation is cut off at about 30%.** `PopupScores.prefab` carries `flyupText.anim`, which is **1.0 s** long — but `LevelManager.cs:2232` does `Destroy(poptxt, 0.3f)`. The fly-up is killed mid-flight.

10.5 The board entrance blocks input for a full second

`LevelManager.AnimateField()` builds a clip **in C# at runtime** and adds an animation event at **t = 1.0** calling `EndAnimGamField`, which sets `gameStatus = Playing`. The motion itself settles at 0.7–0.8 s.

> **So there is roughly 200 ms of dead air at every level start** where the board has visibly stopped moving but the player still cannot touch it. CONFIRMED.

10.6 The original motion ran at 50 Hz

The original gameplay motion coroutines step on `WaitForFixedUpdate()` — the physics tick, **50 Hz** by default (`Fixed Timestep: 0.02`) — while the game renders at 60 fps.

> **Designer translation.** Motion updated 50 times a second but drawn 60 times a second produces uneven steps — a subtle judder. This is a root cause of the original swap and fall feeling less smooth than a modern title, independent of easing or duration. At least 13 `WaitForFixedUpdate` sites remain in the original code paths, including the tutorial hand.

---

11. Existing Animation Library

Full inventory in `08_Animation_Inventory.csv`. Summary of coverage against standard match-3 feedback beats:

Beat	Status	Implementation
Piece idle "breathing"	EXISTS	Coroutine on the root; randomised speed per piece
Piece selection / highlight	**MISSING**	No visual response to touching a fruit
Drag follow	**MISSING**	The piece does not follow the finger; the swap fires on threshold
Valid swap	EXISTS	Coroutine, eased, 130 ms *(AI-edited; was linear ~200 ms)*
Invalid swap	EXISTS	Push-and-return, ~220 ms *(AI-edited; was a full 400 ms round trip)*

Anticipation before pop	EXISTS *(AI-added)*	25 ms
Squash / stretch	EXISTS *(AI-added)*	35 ms squash, 25 ms rebound
Destruction	EXISTS	Originally `destroy.anim` 250 ms; now code-driven 160 ms
Spawn / refill	PARTIAL	Spawns 3 units above and falls; no scale-in
Falling	EXISTS	Originally a lerp ramp; now true velocity integration
Landing	EXISTS	Originally `stop.anim` 258 ms; now a 100 ms code squash
3 / 4 / 5-match differentiation	**MISSING in the original**	The original played the identical puff for every match size
Cascade / combo escalation	PARTIAL	A praise word appears after 3+ waves; no escalating motion
Special-piece creation	EXISTS	`appear.anim`, 333 ms, with a `SetAppeared` event
Special-piece activation	EXISTS	Stripe beam, package explosion, colour-bomb clear
Button press	PARTIAL	26 stock colour tints, 19 Animator clips, **`Highlighted` unreachable**
Objective / goal collection	EXISTS	Runtime-built curve; sprite copy flies to the HUD icon and spins
Score popup	**BROKEN**	1 s clip destroyed at 0.3 s
Popup entrance / exit	EXISTS	`banner.anim` 3.03 s with a **hard-coded 1.6 s dwell baked into the clip**
Level start	EXISTS	Board slides in; **input blocked for 1.0 s**
Win / lose	EXISTS	`bannerComplete` / `bannerFailed`, 3.03 s each
Character reaction	**MISSING**	No character exists in the live game (see 9.6)
Camera shake / punch / zoom	**MISSING**	No camera feedback of any kind

> **On popup dwell:** `banner.anim` moves the banner from y=1000 to y=-8 by t=0.867 s, holds until t=2.467 s, then exits by t=3.0 s. **The 1.6 s hold is baked into the animation file, not driven by a dismiss.** Speeding up a popup means editing the clip.

---

12. VFX Architecture

12.1 Three unconnected systems

Layer	What it is	Origin
A. Sprite-sheet flip-books	One flat image swapped frame by frame, like a flip-book	Original kit — **this is the core match effect**
B. Unity Particle Systems	The real particle engine	Original kit, but only 6 prefabs, of which **3 are wired up**
C. Procedural additive sparkles + bloom	Code-generated textures, 4 pooled particle systems, 96 sprite overlays	**ADDED BY AI AGENT**

12.2 The original had almost no particles

Six original prefabs contain a `ParticleSystem`: `Firework`, `Replace`, `Snow`, `PackageExpl`, `Splash`, `Splash 1`. Of these:

- `Firework` — **wired up** (package explosion, boosters)
- `Replace` — **wired up** (colour-bomb clear), but half of it renders behind the fruit (see Section 8)
- `Snow` — assigned in the scene but the field is **never read by code**; dead
- `PackageExpl`, `Splash`, `Splash 1` — **orphans**, zero references

Everything else the player sees during core match-3 play in the original build is sprite-based, not particle-based.

12.3 Absent in the original — all CONFIRMED

VFX Graph, Shader Graph, custom shaders, post-processing, bloom, screen flash, camera shake, distortion, vignette, chromatic aberration, trails. A grep for `shake|bloom|distort|vignette|postprocess` across all gameplay scripts returns nothing relevant.

12.4 The added layer, and its defects

`MatchVFXService` creates a `DontDestroyOnLoad` singleton with **four shared world-space ParticleSystems** (`Stars` 320, `Micro` 420, `Fragments` 260, `CenterStars` 160 max particles), a **96-entry sprite-overlay pool** and a **128-entry scheduler**. Textures are **generated by maths at load time** — no new art assets were added. VFX

draw at sorting order **25**, i.e. **in front of everything on the board** — the opposite of the original flip-book's order 0, and the main reason the new feedback reads stronger.

Three CONFIRMED defects in this added layer:

1. **The bloom cannot render.** `Assets/Settings/JG_2DRenderer.asset:57` has `m_PostProcessData: {fileID: 0}`. URP's `Renderer2DRendergraph.cs:156` sets `supportedRenderingFeatures.postProcessing = m_Renderer2DData.postProcessData != null`, and line 136 only builds the post-process pass under the same null check. **The `Bloom` volume the code creates at runtime is never rendered.** Unity normally assigns this resource automatically when a 2D Renderer is created through the editor's Create-Asset menu; this asset was created programmatically and bypassed that.

2. **The shader is not SRP-Batcher compatible.** `_Color`, `_Tint` and `_Boost` are declared as bare uniforms outside any `UnityPerMaterial` CBUFFER.

3. **The shader multiplies alpha twice.** It sets `Blend SrcAlpha One` *and* does `col.rgb = col.a _Boost` in the fragment. True premultiplied-additive would be `Blend One One`. The practical effect is that sparkles fade faster and dimmer than the boost value implies, and tuning that value behaves non-linearly.

---

13. Existing VFX Library

13.1 The original core match effect — the key finding

`Assets/JellyGarden/Resources/Prefabs/Effects/ItemExpl.prefab`:

- Components: `Transform`, `SpriteRenderer`, `Animator`, `ItemAnimEvents`. **No ParticleSystem.**
- `m_SortingOrder: 0`, material Sprites-Default (plain alpha, **not additive**)
- Controller `ItemDestroyController.controller` with an **Int parameter `color`** selecting one of six clips
- `item_anim1.anim`: sample rate 15, **4 visible frames over about 267 ms**, then a blank frame; `m_Events: []`
- Frame art: 163 x 163 px at 100 PPU, drawn at `localScale 0.5` = **about 0.82 world units**, i.e. exactly one cell

> **Designer translation — why the original destruction reads flat.**

> - It is **four hand-drawn puff images played like a flip-book**. Nothing scatters, nothing has gravity, nothing inherits the fruit's motion. Every 3-match of a given colour looked **byte-for-byte identical**, always 267 ms, always the same shape.

> - **It draws at sorting order 0 while the fruit draws at order 2 — the puff is *behind* the piece it is destroying**, and behind every neighbouring fruit it overlaps.

> - There are only **six colour variants** and **no variation in size, rotation, intensity or count**. A 3-match, a 4-match and a 5-match produced exactly the same puff.

> - **It is now dead code.** With `EnableJuice: 1` the destruction routes through the new path. The 20 pooled flip-book objects are still created every session and never used.

13.2 Pooling in the original

Exactly **one** pool exists: `LevelManager.itemExplPool`, 20 `ItemAnimEvents` created in `Awake`. `GetExplFromPool()` linear-scans for a free slot and **returns null when exhausted**, in which case the caller silently plays no effect. Each slot is held for a **full 1.0 s** even though the visual is 267 ms — so a cascade destroying more than 20 fruits in a second **silently loses effects on the extras, precisely on the biggest clears**.

Everything else is `Instantiate`/`Destroy` per use: `Firework`, `Replace`, `CircleExpl`, booster overlays, `StripesEffect`, score popups, blocker debris.

13.3 Coverage against expected match-3 feedback

Effect	Original	Now
3-match burst	PARTIAL — flip-book behind the fruit	EXISTS — layered particle burst in front
4-match / 5-match differentiation	**MISSING**	PARTIAL — intensity scales with size and cascade depth
Impact flash	**MISSING**	EXISTS
Sparkle / star burst	**MISSING**	EXISTS
Cascade escalation	**MISSING**	PARTIAL
Special-piece creation	EXISTS (`appear.anim`)	unchanged
Stripe activation	EXISTS (`StripesEffect`)	unchanged
Package explosion	EXISTS (`Firework`, particles)	unchanged
Colour-bomb clear	EXISTS (`Replace`) — **half of it behind the fruit**	unchanged
Blocker destruction	PARTIAL	unchanged
Landing dust	**MISSING**	**MISSING**

Trails	**MISSING**	**MISSING**
Goal collection	EXISTS (sprite flies to HUD)	unchanged
Win celebration	EXISTS (stars, banner)	unchanged
Screen flash / camera shake	**MISSING**	**MISSING**
Bloom / glow	**MISSING**	**PRESENT BUT NON-FUNCTIONAL** (12.4)

---

14. Audio Architecture

14.1 One AudioSource, one method, no mixer

The entire sound system is two small singleton scripts. Every sound in the game goes through:

```
public void PlaySound(AudioClip clip) {
    if (_audioSource && clip) _audioSource.PlayOneShot(clip);
}
```

There is no AudioMixer asset anywhere in the project, and both AudioSources route straight to the master output (`OutputAudioMixerGroup: {fileID: 0}`). CONFIRMED.

> **Designer consequences, all of them significant:**

> - **You cannot duck the music** under a win stinger.

> - **You cannot compress or limit the SFX bus**, so a five-fruit cascade firing eight overlapping sounds simply sums — and can clip.

> - **Volume is binary and global.** Two on/off toggles (`Sound`, `Music`) in the pause menu. **There is no volume slider.**

> - **There is zero stereo width.** Both sources weld themselves to the camera at `Awake`, and the `AudioListener` is on that same camera, so distance is zero and everything plays dead-centre. Board-position panning — standard polish in a premium match-3 — is not merely unimplemented; the single-shared-source architecture makes it impossible without a rewrite.

> - `m_DSPBufferSize: 0` = "Best performance" — **the largest buffer and therefore the highest audio latency of the three presets**, in a game where the sound must land on the visual pop.

14.2 The files are AIFF, not WAV

All 35 files in `Assets/JellyGarden/Audio/wav/` begin `FORM....AIFF` — they are **Apple AIFF containers with a `.wav` extension**, 16-bit / 44.1 kHz. **25 are mono, 10 are stereo.** CONFIRMED from the COMM chunk.

Music: `game_map_music.wav` is **about 172 s (2:52)** and `game_music.wav` about **53 s** — both **mono**, and both are the largest memory items by an order of magnitude.

Every one of the 35 files has identical import settings: `loadType: 0` (Decompress On Load), `compressionFormat: 1` (Vorbis), `forceToMono: 0`, and — materially — **`platformSettingOverrides: {}`**, i.e. **zero per-platform audio compression overrides on a mobile title.**

> Decompress-On-Load means each clip is fully decompressed into RAM at load. For a 172 s mono track that is a large, avoidable allocation; `Streaming` is the normal choice for music.

14.3 Event coverage

Full mapping in `10_Audio_Inventory.csv`. Notable findings:

- **Random variation exists in exactly two places**: `destroy[0..1]` (two jelly-destroy clips) and `drop[0..1]`. Everything else plays a fixed clip.
- **`.pitch` is never touched anywhere in the codebase.** A grep across all `.cs` returns nothing. **There is no pitch variation and no rising cascade ladder.**
- **One escalation cue does exist:** the win sequence plays `star[0]`, `star[1]`, `star[2]` in ascending order with 0.5 s between them for a 3-star win. That is the game's only rising ladder — and the model to copy for cascades.
- **A wiring defect:** `destroyPackage` and `explosion` point at the **same clip** (`Explosion.wav`), while `Package_destroy.wav` (226 KB) sits on disk referenced by nothing. **The wrapped-package detonation plays the generic explosion instead of its own sound.**
- **`Combo 01.wav` exists and is wired to nothing** — the exact asset a cascade-escalation system would need.
- Two clips are never called: `alert` (only a commented-out call site) and `gameOver[1]`.

Missing audio feedback: no piece-selection sound, no cascade escalation, no combo sound, no landing sound distinct from the drop, no per-match-size differentiation.

---

15. Existing Audio Library

35 clips, roughly 20 MB on disk (dominated by the two music tracks at 14.5 MB and 4.5 MB). About **747 KB is orphaned**. Full inventory in `10_Audio_Inventory.csv`.

---

16. Camera / Lighting / Rendering

- **Camera:** one orthographic camera on `Main Camera`, saved `orthographicSize: 8.61`, overridden at runtime to 6.5 / 7.6 / 8.1 by aspect bucket (see 7.3). `m_HDR: 0` in the scene; the added bloom code flips `allowHDR = true` at runtime.
- **`AspectCamera.cs`** exists and adjusts the camera, and contains a no-op (`Mathf.Clamp(maxLength, 4, maxLength)`) and an unused variable — a sign of the code's maturity level.
- **Lighting: none.** Zero `Light` components and zero `Light2D` in either scene. The art is fully pre-lit; nothing is dynamically shaded. CONFIRMED.
- **Shadows:** none.
- **Sorting layers:** only two, `Default` and `UI` (see 7.3).
- **Post-processing:** absent in the original; present but non-functional now (12.4).
- **Camera stacking / Cinemachine:** not used.

16.1 A trap for anyone adding new sprite art

URP's 2D pipeline sets its **default sprite material to `Sprite-Lit-Default`**. This project has **zero `Light2D` objects**.

> **Designer warning.** Existing art is safe — every existing `SpriteRenderer` has `Sprites-Default` (the unlit built-in material) explicitly assigned, and there are 1,035 such references across the prefabs. But **any *new* sprite created in the scene will default to the lit material and, with no 2D lights present, is likely to render black.** This is the most probable thing to bite an artist or designer adding art to this project today, and the fix is simply to assign the unlit material.

---

17. Materials & Shaders

Only **8 materials** exist in the whole project, and the original used **no custom shaders at all**.

Material	Shader	Used by
`Materials/package_particle.mat`	`Mobile/Particles/Alpha Blended`	`Firework`
`Materials/Bubble.mat`	`Legacy Shaders/Particles/Additive (Soft)`	`Replace`
`Materials/ElectricBolt_Add 1.mat`	`Legacy Shaders/Particles/Additive`	`Replace`
`Fonts/Materials/snow.mat`	`Legacy Shaders/Particles/Additive`	`Snow` (dead)
`Fonts/Materials/Water/*.mat` (3)	`Legacy Shaders/Particles/Additive (Soft)`	orphaned
`Resources/JuiceAdditive.mat`	`JellyGarden/JuiceAdditive`	**ADDED BY AI AGENT**

Every sprite in the game — fruit, cells, blockers, all UI Images — uses Unity's stock **Sprites-Default**. CONFIRMED: 1,035 references across prefabs.

These legacy particle shaders are **Built-In pipeline shaders now running inside a URP project**. They compile and are supported, but they are not SRP-Batcher compatible.

17.1 Where each visual effect actually comes from

This is the question a designer most needs answered, and here it is for this specific project:

Visual	Source in THIS project
Fruit glossy highlight	**SOURCE IMAGE** — painted into the PNG
Candy-style text stroke	**DUPLICATED GEOMETRY** — the `Outline` component makes 4 shifted copies of the text mesh
Button pressed look	**MATERIAL/VERTEX TINT** (26 buttons) or **ANIMATION** (19 buttons)
Board checkerboard	**SOURCE IMAGE** — two alternating sprites
Jelly / blocker look	**SOURCE IMAGE**
Destroy puff (original)	**SPRITE-SHEET ANIMATION** — 4 frames, Animator-driven
Package explosion	**PARTICLE SYSTEM** (`Firework`)
Colour-bomb clear	**PARTICLE SYSTEM** (`Replace`, 2 systems)
Stripe beam	**PREFAB + ANIMATION** (`StripesEffect`)
Score number float	**LEGACY ANIMATION CLIP** (cut off at 30%)
Objective fly-to-HUD	**RUNTIME-GENERATED CURVE** in C#
Board entrance	**RUNTIME-GENERATED CLIP** in C#
Sparkle burst *(added)*	**PARTICLE SYSTEM + PROCEDURAL TEXTURE + CUSTOM SHADER**
Glow / bloom *(added)*	**POST-PROCESSING — currently non-functional**

Nothing in this project uses a shader for glow, outline, dissolve, greyscale, blur, distortion or colour replacement. Every such look, where it exists, is painted into the source image.

---

18. Third-Party Packages

Full inventory in `12_Package_Inventory.csv`. Fifteen explicit non-module packages plus 34 Unity modules.

Package	Version	Status
`com.unity.ugui`	2.5.0	**IN ACTIVE USE** — the entire UI
`com.unity.inputsystem`	1.20.0	**IN ACTIVE USE** — `activeInputHandler: 1`
`com.unity.render-pipelines.universal`	17.5.0	**Installed and shipped unused**; now activated by the AI agent
`com.unity.purchasing`	5.0.2	INSTALLED — gated behind `UNITY_INAPPS`, **not defined on Android/iOS**
`com.unity.timeline`	1.8.13	INSTALLED, **no `.playable` asset exists — unused**
`com.unity.visualscripting`	1.9.12	INSTALLED, no evidence of use
`com.unity.ai.*` (assistant, inference, navigation)	various	INSTALLED, no evidence of use in a 2D match-3
`com.unity.multiplayer.center`	1.0.1	INSTALLED, unused
`com.unity.collab-proxy`	2.13.6	Version control tooling
`com.unity.test-framework`	1.7.0	INSTALLED, **no tests exist**

Third-party code inside `Assets/` (not packages):

Library	Location	Status

Malee ReorderableList	`Assets/JellyGarden/Unity-Reorderable-List-master/`	**IN ACTIVE USE** — `TargetLevel.cs` uses `[Reorderable]`; it is what makes the level-objective inspector editable
Google Mobile Ads	`Scripts/GoogleRewardedAds/`	Behind `GOOGLE_MOBILE_ADS`, **not defined** — dormant
PlayFab / GameSparks	`Scripts/Integrations/Network/`	Behind undefined symbols — dormant
Facebook	`Scripts/Integrations/FacebookManager.cs`	Behind `FACEBOOK`, not defined — dormant

18.1 A likely build-blocking configuration problem

`UNITY_ADS` **is** defined for Android, iPhone and Windows Store. Four files contain `using UnityEngine.Advertisements;` inside `#if UNITY_ADS`. But **there is no Unity Ads package in `manifest.json` or `packages-lock.json`.**

> **This is a near-certain compile failure on every shipping platform.** It is not a subtle risk — the define is on, the `using` is compiled, and the assembly it names is absent. INFERRED (I did not run a platform build), but the evidence is unambiguous. It should be verified before any production work is scheduled.

Separately, the scripting defines were changed at some point before this audit (not by the AI agent): **Android lost `UNITY_INAPPS`** and **iOS/WSA lost `GOOGLE_MOBILE_ADS`**. Since `UNITY_INAPPS` gates real code in `AnimationManager.cs`, **in-app purchases are compiled out of Android builds as configured.**

---

19. Performance Observations

Observation only — nothing was optimised.

19.1 Confirmed problems

Issue	Evidence	Impact
`GameObject.FindGameObjectsWithTag("Item")` called about **10x/second during every cascade**, in four separate methods, each allocating a ~99-element array plus a `GetComponent` per element	`IsAllDestoyFinished`, `IsAllItemsFallDown`, `GetRandomItems`, `Item.DestroyColor`	**The primary GC/CPU risk, and it peaks during the most visually busy moments**
No Sprite Atlases	zero `.spriteatlas` files	Draw-call count far higher than necessary

`Item.Update()` runs `Physics2D.OverlapPoint` **per fruit per touch-frame**	`Item.cs:1403-1436`	Up to ~99 redundant physics queries for one tap, duplicating `LevelManager`'s own query
Almost nothing is pooled	one 20-slot pool; everything else `Instantiate`/`Destroy`	Steady GC pressure and intermittent hitches
53 `Outline` components, each quadrupling its text's triangles	`game.unity` census	Avoidable UI fill and vertex cost
Music is Decompress-On-Load	all 35 `.meta` files	A 172 s track fully decompressed into RAM
No per-platform texture or audio overrides	all `.meta` files	Ships desktop-grade assets to phones
18 redundant `Resources.Load` calls per level load	`LevelManager.cs:2379` is inside the line-parsing loop	Load-time only, but a clear code-maturity signal

19.2 Risks (not confirmed as problems)

- The added VFX layer budgets **1,160 particles** across four additive systems plus 96 additive sprite overlays. Additive transparency is fill-rate-bound on mobile; this has **not** been profiled on a device.
- The two legacy Built-In particle shaders and the added custom shader are all **SRP-Batcher incompatible**.
- 192 sprites are non-power-of-two, which limits some compression formats.

19.3 What is genuinely good

The added `MatchVFXService` is well-built for mobile: zero per-match allocation, shared world-space particle systems, fixed-size pools, and a scheduler that avoids per-effect coroutines. It is the one system in the project that follows current Unity pooling guidance.

---

20. Existing Visual Design Language

20.1 What is consistent and intentional

- **A single house typeface**, `Candy Script`, on 51 of 57 in-scene labels and 105 prefab labels. This is the strongest consistency in the project.

- **A warm candy palette** — the recurring outline colours cluster on a warm brown (0.400, 0.157, 0.051) and a cream (0.996, 0.847, 0.569).
- **A consistent frame idiom** — wooden/candy plates with rounded corners for HUD and popups.
- **Consistent piece silhouettes** — six fruit shapes at a uniform 128 x 128, all with a painted top-left gloss highlight.
- **Backgrounds rotate every 20 levels**, giving deliberate long-arc variety.

20.2 What is inconsistent or ad hoc

- **Two reference resolutions** (720 x 1280 for HUD, 800 x 1280 for popups), so popups render about 11% smaller relative to the screen.
- **Seven authored font sizes**, each further modified by an arbitrary per-object scale, and two of them overwritten at runtime. There is no type scale.
- **About 18 distinct outline configurations** with offsets like `5.01, -2.49` and `6.02, -5.81` — the signature of dragging a slider rather than choosing a value.
- **A non-uniform Z scale** on the Score label (0.6915 vs 0.3457) — meaningless, but evidence of hand-dragging.
- **Two typefaces beyond the house font**, one used only on inactive objects.
- **Word art is baked into PNGs** ("SCORE", "MOVES", "TARGET", "LEVEL") — these cannot be localised or restyled without repainting.
- **A second, better outline component exists but is unused.**
- **Three water/snow particle materials live in a `Fonts/` folder.**

20.3 Animation timing language

There is no consistent timing language. Observed durations: 250 ms (destroy), 258 ms (landing), 333 ms (special appear), 267 ms (destroy puff), 1.03 s (button idle, unreachable), 3.03 s (every banner), 1.0 s (score popup, cut to 0.3 s). Easing is almost entirely **linear** in the original code paths.

> **Designer summary.** The *art direction* is coherent — one typeface, one palette, one frame idiom. The *implementation* is not: sizes, outlines, timings and easings were each decided locally. There is a real design language here, but it lives in the artwork rather than in the project's structure, and nothing enforces it.

---

21. Match-3 Player Feedback Experience Map

One normal 3-match, traced through every feedback layer. See `15_Match3_Feedback_Experience_Map.md` for the full version.

Stage	Visual	Motion	VFX	Sound	Camera	Gap
Touch a piece	—	—	—	—	—	**No selection feedback at all**

Drag	—	Piece does not follow the finger	—	—	—	**No drag affordance**
Swap (valid)	pieces exchange	eased, 130 ms	—	—	—	—
Swap (invalid)	push and return	~220 ms	—	`Wrong_match.wav`	—	—
Match confirmed	—	—	—	—	—	**100 ms of nothing** (hard wait)
Anticipation	slight pull-in	25 ms	—	—	—	*(added)*
Squash	X 1.07 / Y 0.92	35 ms	—	—	—	*(added)*
Impact	flash	—	flash + stars + micro + fragments	`Jelly_destroy` (random of 2)	—	Original had **no flash**
Piece clear	scale-up + fade	75 ms	burst continues	—	—	Original: 250 ms shrink, puff **behind** the piece
Board settles	—	—	—	—	—	**The gate: everything waits for the slowest death**
Fall	pieces drop	velocity integration	—	`drop1/2` on landing	—	—
Landing	squash	100 ms	—	—	—	**No landing dust**
Refill	spawn 3 units above	falls in	—	—	—	**No spawn scale-in**
Cascade	repeat	—	intensity scales *(added)*	**identical sound every wave**	—	**No audio escalation**
Goal update	sprite flies to HUD	runtime curve + spin	—	`Get_star_ingredient`	—	—

Board ready	—	—	—	—	—	Hint fires 2 s later regardless of player intent

The three biggest experience gaps, in order:

1. **Nothing happens when you touch a piece.** No highlight, no lift, no scale, no sound. The first moment of every interaction is silent and invisible.

2. **Audio does not escalate.** A 3-match and a 7-piece cascade play the same sound at the same pitch. `Combo 01.wav` is sitting unused in the project.

3. **The pipeline's hard waits dominate.** About 0.8 s of every cascade wave is `WaitForSeconds`, not animation — so polish work on the art has a low ceiling until those are addressed.

---

22. Reusable Systems & Assets

These are solid and worth building on rather than replacing:

System	Why it is reusable	Location
Level format	Plain text, one file per level, human-readable and diffable	`Resources/Levels/1..100.txt`
Level editor	A complete 85 KB in-Editor authoring tool for that format	`Scripts/Editor/LevelMakerEditor.cs`
Objective ScriptableObjects	100 data assets, editable in the Inspector via the Malee reorderable list	`Resources/Targets/Level1..100.asset`
Booster dispatch	One clean property setter routes all eight booster types	`LevelManager.ActivatedBoost`
Special-piece combination matrix	Five distinct pair behaviours, all implemented and working	`Item.cs:629-703`
The world map	Spline-based path, camera pan, per-level nodes and stars	`Scripts/MapScripts/`
The Item prefab's two-level rig	Root for code motion, child for authored clips — genuinely well-suited to layered animation	`Resources/Prefabs/Item.prefab`

`JuiceConfig.asset` *(added)*	~45 tuning sliders as data — the only way to iterate on feel without a recompile	`Resources/JuiceConfig.asset`
`MatchVFXService` *(added)*	Pooled, allocation-free VFX with procedural textures	`Scripts/Juice/`
Fruit and blocker art	Consistent 128 x 128 silhouettes, coherent palette	`Textures/Items/`, `Textures/blocks/`
`GameRedesign/` concept art	An existing mascot with six expressions and four HUD treatments	`Assets/GameRedesign/`

---

23. Missing / Weak Systems

Missing	Severity	Note
Piece selection feedback	**Critical**	The first moment of every interaction has no response
Sprite Atlases	**High**	Pure import-settings work; no art changes needed
9-slice borders	**High**	Blocks all resizing of chrome
TextMeshPro	**High**	Blocks crisp scaling, gradients and real outlines
Audio mixer + cascade escalation	**High**	No ducking, no pitch, no combo ladder
Localization	**High**	Plus word art baked into PNGs
Camera feedback (shake / punch)	Medium	None of any kind exists
Landing dust / spawn scale-in	Medium	Fall and refill have no arrival accent
Match-size differentiation in the original VFX	Medium	Partly addressed by the added layer
Scroll views / masks / canvas groups	Medium	Limits any list or panel redesign
Object pooling beyond one 20-slot pool	Medium	Steady GC pressure

Per-platform texture/audio overrides	Medium	Ships desktop-grade assets to phones
Automated tests	Low	Test framework installed, zero tests
Cloud save	Low	PlayerPrefs only

---

24. Technical Constraints Relevant to UI/UX

These are the constraints to design *around*. Each is CONFIRMED.

1. **The board and the HUD scale by different rules** and are only tuned for three device aspect ratios. Any layout must tolerate the gap between them varying by device. (7.3)

2. **Feedback timing is bounded by hard-coded waits**, floor about 0.8 s per cascade wave. Animation polish has a low ceiling until those change. (5.4)

3. **The destroy gate rounds up to 100 ms.** Animation durations of 190 ms and 260 ms cost 200 ms and 300 ms. There are no in-between values. (5.4)

4. **Earning a special piece costs a flat extra 500 ms**, and each special in a destroyed group costs another 100 ms.

5. **Nothing can be 9-sliced**, so no chrome can be resized without a new PNG. (7.6)

6. **Nothing on the board can draw over the HUD** — they are on different sorting layers. (7.3)

7. **Text does not auto-size and has no TMP features.** Long strings overflow. (7.4)

8. **Font size in the Inspector is not the size on screen**, and two labels are rescaled at runtime. (7.4)

9. **Some labels are baked word art** and cannot be localised or restyled without repainting.

10. **There are no layered source files** — every UI asset is a flattened PNG. (9.7)

11. **Animation clip lengths are gameplay timing values** because of animation events. (10.2)

12. **Popup dwell time is baked into the clip**, not driven by a dismiss. (11)

13. **Input is blocked for a full second at level start**, about 200 ms of it after motion has stopped. (10.5)

14. **The swipe threshold is in world units**, so sensitivity differs per device. (5.1)

15. **The hint fires on a fixed 2 s clock**, not on player idleness. (5.6)

16. **A new sprite added to the scene will likely render black** under URP without an explicitly assigned unlit material. (16.1)

17. **Everything lives in one 787 KB scene** — concurrent editing causes merge conflicts. (2.2)

18. **No localization system, and no bundle identifier is set.** Both are shipping blockers.

19. `Assets/GameRedesign/Resources/` **will ship in the build** despite being referenced by nothing. (9.6)

---

25. External Best-Practice Comparison

Separated strictly into what this project does and what external sources recommend. See `17_External_Best_Practice_References.md` for sources.

Topic	PROJECT FACT	EXTERNAL BEST-PRACTICE REFERENCE
Text	Legacy `UnityEngine.UI.Text`, ~167 components, outlines faked by 4x geometry	Unity documents TextMeshPro as "the ultimate text solution and the perfect replacement for Unity's UI Text", noting the visual gain comes "at no additional performance cost" since both use two triangles per character
Atlasing	Zero Sprite Atlases; legacy packing tags only	Unity's guidance is to keep sprites drawn together on one atlas, since multiple atlases per object "results in multiple draw calls"
Post-processing	2D Renderer created with `postProcessData` null, so bloom cannot run	Unity documents that Unity "automatically assigns a default Asset to the post-processing data property" when a renderer is created through the editor
Impact feel	No hit-stop in the original; one added at 45 ms	Industry writing on game feel describes a micro-pause of **40–80 ms** as selling weight better than additional animation
Match-3 pacing	Roughly 0.8 s of hard waits per cascade wave	Analyses of Royal Match note it is "the fastest switcher with quick and controlled animations", and that players "continue progressing without needing to wait for special effects to complete"
Match VFX	Original was a 4-frame flip-book behind the piece	Unity's own Gem Hunter Match 2D sample demonstrates URP 2D lighting plus a Radial Warp shockwave shader for match impact
Audio	One AudioSource, no mixer, no pitch variation	Standard practice is a mixer with SFX/Music groups for ducking and limiting, and pitch variation on repeated SFX to avoid ear fatigue
Pooling	One 20-slot pool; everything else Instantiate/Destroy	Pooling frequently spawned objects is long-standing Unity mobile guidance

Not claimed: none of the external recommendations describe something this project already does. The added `MatchVFXService` is the sole system that independently matches current pooling guidance.

---

26. Prioritized Gap Analysis

Full version with evidence in `16_Gap_Analysis.md`.

CRITICAL

Gap	Evidence
`UNITY_ADS` is defined on all three shipping platforms but no Unity Ads package is installed	Four files `using UnityEngine.Advertisements;` inside `#if UNITY_ADS`; absent from `manifest.json` and `packages-lock.json`. Near-certain compile failure on Android/iOS/WSA
No piece-selection feedback	No highlight/scale/sound on touch anywhere in `Item.cs`
Cascade pacing floor of ~0.8 s per wave is hard-coded	Four `WaitForSeconds` literals + 100 ms polling in `FallingDown()`

HIGH

Gap	Evidence
Board and HUD scale by different rules, tuned for only 3 aspect ratios	`LevelManager.cs:411-443` exact float equality; `matchWidthOrHeight: 0` on all canvases
The added bloom cannot render	`JG_2DRenderer.asset:57 m_PostProcessData: {fileID: 0}`; URP gates the pass on that field
No Sprite Atlases	Zero `.spriteatlas` files; 225 sprites
No 9-slicing anywhere	All 225 `spriteBorder` are zero
No TextMeshPro	Zero TMP assets; 57 legacy Texts in-scene, ~167 project-wide
Audio has no mixer, no pitch, no cascade escalation	No `.mixer` asset; `.pitch` never referenced

Popup and HUD reference resolutions disagree by 11%	800x1280 vs 720x1280, both match-width
No localization; word art baked into PNGs	No localization package; `score_txt.png` and similar
Safe area applied to one object only, and does not survive rotation	One `SafeArea.cs` reference; rotation is enabled

MEDIUM

Gap	Evidence
Score popup animation destroyed at 30% of its length	1.0 s clip, `Destroy(poptxt, 0.3f)`
Button `Highlighted` and `button_idle` states are unreachable	`button_controller.controller` transition lists
`Replace.prefab`'s Bubble half renders behind the fruit	Child renderer order 0 vs fruit order 2
Package detonation plays the generic explosion	`destroyPackage` and `explosion` share a GUID; `Package_destroy.wav` unused
1.0 s input lock at level start, ~200 ms of it dead air	`AnimateField` event at t=1.0; motion settles at 0.8
~10 `FindGameObjectsWithTag` calls/second during cascades	Four call sites
Almost nothing is pooled	One 20-slot pool
Added shader is SRP-Batcher incompatible and double-multiplies alpha	Bare uniforms; `Blend SrcAlpha One` plus `col.rgb *= col.a`
38 MB of orphaned art, 21 PNGs of it in a `Resources` folder	Zero references; force-packed into builds
`Assets/GameRedesign/Resources/HUD_Card.png` is the project's only 9-sliced sprite	`spriteBorder: {165,140,165,185}`

LOW

Misfiled particle materials in `Fonts/`; ~747 KB orphaned audio; `item_01.controller` and `destroyItem.anim` orphaned; `UNDESTROYABLE` blocker type unused by all 100 levels; `demo` scene referenced but not in build

settings; `Assets/_Recovery/0.unity` is a byte-for-byte content duplicate of `game.unity` differing only in line endings.

---

27. What Must Be Understood Before Redesign

1. **This project is not in its shipped state.** Decide first whether to keep, revert or repair the AI agent's changes — particularly the URP migration, which is live but has a non-functional bloom. Everything else in the report describes the original kit.

2. **The board is not UI, and that is not changeable cheaply.** Any mockup must be tested against the fact that board and HUD scale independently, and that only three device aspect ratios are tuned.

3. **Feedback work has a ceiling until the pipeline waits are addressed.** Roughly 0.8 s of every cascade wave is hard-coded waiting. Beautiful 120 ms animations will still feel slow inside a 0.8 s wave. **This is the single highest-leverage engineering change for perceived quality**, and it is a change to `LevelManager.cs`, not to art.

4. **Three import-settings changes would deliver disproportionate value** and require no art rework: create Sprite Atlases, add 9-slice borders to the chrome sprites, and add per-platform texture/audio overrides.

5. **There are no editable source files.** Any visual change means repainting flattened PNGs. Budget for rebuilding source art.

6. **Some text is not text.** "SCORE", "MOVES", "TARGET" and "LEVEL" are images. Any typographic or localization work must replace them.

7. **The strongest existing foundation is the data layer** — the text level format, the level editor and the objective ScriptableObjects. A redesign can safely rebuild presentation on top of it.

8. **`Assets/GameRedesign/` is prior art worth reading before starting.** It contains a mascot with six emotional states and four HUD concepts, none of it wired up. Whoever made it had already begun answering some of the same questions.

9. **Verify the ads build failure before scheduling anything.** If `UNITY_ADS` genuinely breaks mobile compilation, no build exists to iterate against.

Prepared from direct inspection of project files, serialized Unity YAML, `.meta` importer data, git history and package manifests. Every material claim is labelled CONFIRMED, INFERRED or NOT CONFIRMED at its point of use. No file inside `Assets/`, `Packages/`, `ProjectSettings/` or `UserSettings/` was modified to produce this document.